

CHATBOT WITH RULE-BASED RESPONSES

Here, it is explained the code in my healthcare chatbot project step by step. The project has two main code files: chat_bot.ipynb (a Jupyter notebook with cells) and chat_bot.py (a plain Python script). They contain almost the same code, but the notebook breaks it into smaller sections with explanations. I'll focus on the notebook since it's more structured, and reference the script where it differs.

This is a chatbot that uses machine learning (ML) to predict diseases based on symptoms you input. It combines rule-based logic (like if-then rules) with ML models like Decision Trees and Support Vector Machines (SVM). It also uses data from CSV files for symptoms, descriptions, and precautions.

We will go through each "section" (cell) in the notebook, explaining what it does, why it's there, and key concepts for beginners. Will keep it simple—no deep math or code jargon unless needed.

1. Title Cell (Markdown)

Code/Content: ``# HealthCare ChatBot ( Rule-Based + ML algorithms)``

Explanation: This is just a title. It tells you what the project is: a healthcare chatbot that uses rules (like simple logic) and machine learning algorithms to work. Markdown cells like this are for notes and headings—they don't run as code.

2. Import Libraries Cell (Markdown)

Code/Content: ``# Import Libraries``

Explanation: Another heading. Libraries are pre-written code packages that add features to Python. The next cell imports them.

3. Import Libraries (Code Cell)

Code:

```
```python
import re
import pandas as pd
import pytsx3
from sklearn import preprocessing
```

```
from sklearn.tree import DecisionTreeClassifier, _tree
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.model_selection import cross_val_score
from sklearn.svm import SVC
import csv
import warnings
warnings.filterwarnings("ignore", category=DeprecationWarning)
'''
```

**Explanation: This brings in tools from libraries:**

- ``re``: For searching text patterns (like finding symptoms in a list).
- ``pandas`` (as ``pd``): For handling data tables (like Excel sheets, but in code).
- ``pyttsx3``: For text-to-speech (making the chatbot "speak" if needed).
- ``sklearn`` stuff: Machine learning tools from scikit-learn. ``preprocessing`` helps prepare data, ``DecisionTreeClassifier`` is an ML model that makes decisions like a flowchart, ``train_test_split`` splits data for training/testing, etc.
- ``numpy`` (as ``np``): For math on numbers and arrays.
- ``csv``: For reading CSV files (comma-separated values, like your data files).
- ``warnings``: Ignores annoying warning messages.

This is like gathering your toolbox before building something. Without these, the code couldn't do ML or read files.

#### ### 4. Load Dataset (Markdown)

Code/Content: ``# Load Dataset``

**Explanation: Heading for loading data.**

#### ### 5. Load Dataset (Code Cell)

Code:

```
```python
```

```
training = pd.read_csv('Data/Training.csv')
testing= pd.read_csv('Data/Testing.csv')
cols= training.columns
cols= cols[:-1]
x = training[cols]
y = training['prognosis']
y1= y
...
```

Explanation: Loads your data files.

- `training` and `testing` are tables from CSV files. Training data teaches the ML model; testing checks if it works.
- `cols` gets column names from training data, but removes the last one (`[:-1]` means "all except the last").
- `x` is the input features (symptoms, like columns in the table).
- `y` is the output (the disease prediction, called 'prognosis').
- `y1` is a copy of `y` (saved for later).

Think of it as loading a spreadsheet: rows are patients, columns are symptoms, and one column is the answer (disease).

6. Reduced Data (Code Cell)

Code:

```
```python
reduced_data = training.groupby(training['prognosis']).max()
...
```
```

Explanation: Groups the training data by disease (`prognosis`) and takes the maximum value for each symptom per disease. This creates a "summary" table showing which symptoms are most associated with each disease. It's used later to ask about relevant symptoms.

7. Preprocessing (Markdown)

Code/Content: `# Preprocessing`

Explanation: Heading for preparing data.

8. Label Encoding (Code Cell)

Code:

```
```python
le = preprocessing.LabelEncoder()
le.fit(y)
y = le.transform(y)
```
```

Explanation: ML models need numbers, not words. `LabelEncoder` turns disease names (like "flu") into numbers (like 0, 1, 2). `fit` learns the mapping, `transform` applies it. Now `y` is numbers instead of text.

9. Train-Test Split (Code Cell)

Code:

```
```python
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.33, random_state=42)
testx = testing[cols]
testy = testing['prognosis']
testy = le.transform(testy)
```
```

Explanation: Splits data for training and testing.

- `x\_train`/`y\_train`: Data to teach the model (67% of training set).
- `x\_test`/`y\_test`: Data to check the model (33% of training set).
- `testx`/`testy`: The separate testing file, also encoded to numbers.
- `test\_size=0.33` means 33% for testing. `random\_state=42` makes it repeatable (same split every time).

10. Train Models (Markdown)

Code/Content: `# Train models`

Explanation: Heading for training ML models.

11. Decision Tree Model (Code Cell)

Code:

```
```python
clf1 = DecisionTreeClassifier()
clf = clf1.fit(x_train,y_train)
print(clf.score(x_train,y_train))
print ("cross result=====")
scores = cross_val_score(clf, x_test, y_test, cv=3)
print (scores)
print (scores.mean())
```
```

Explanation: Trains a Decision Tree model.

- `DecisionTreeClassifier()` creates the model.
- `fit` trains it on `x\_train` (symptoms) to predict `y\_train` (diseases).
- `cross\_val\_score` tests it with cross-validation (splits data 3 ways and averages scores).
- Prints the average accuracy (how often it guesses right).

Decision Trees work like flowcharts: "If symptom A, then check B, etc."

12. SVM Model (Code Cell)

Code:

```
```python
model=SVC()
model.fit(x_train,y_train)
print("for svm: ")
print(model.score(x_test,y_test))
```
```

'''

Explanation: Trains a Support Vector Machine (SVM) model.

- ``SVC()`` creates it.
- ``fit`` trains it.
- ``score`` checks accuracy on test data.
- SVM is another ML method, good for classifying data.

It compares to the Decision Tree to see which is better.

13. Feature Importances (Code Cell)

Code:

```
'''python
importances = clf.feature_importances_
indices = np.argsort(importances)[::-1]
features = cols
'''
```

Explanation: From the Decision Tree, gets how important each symptom (feature) is for predictions. Sorts them from most to least important. Used to prioritize symptoms in questions.

14. Text-to-Speech Function (Code Cell)

Code:

```
'''python
def readn(nstr):
    engine = pyttsx3.init()
    engine.setProperty('voice', "english+f5")
    engine.setProperty('rate', 130)
    engine.say(nstr)
    engine.runAndWait()
    engine.stop()
```

'''

Explanation: Defines a function ``readn`` to speak text aloud.

- ``pyttsx3.init()`` starts the speech engine.
- Sets voice to female English and speed to 130 words/min.
- ``say`` speaks the string ``nstr``, ``runAndWait`` waits for it to finish.
- (Commented out in the main code, so it doesn't speak by default.)

15. Dictionaries (Markdown)

Code/Content: ``# Dictionaries``

Explanation: Heading for creating data storage.

16. Initialize Dictionaries (Code Cell)

Code:

```
```python
severityDictionary=dict()
description_list = dict()
precautionDictionary=dict()
'''
```

**Explanation:** Creates empty dictionaries (like key-value stores) to hold:

- ``severityDictionary``: How severe each symptom is (from CSV).
- ``description_list``: Disease descriptions.
- ``precautionDictionary``: Precautions for diseases.

### ### 17. Symptoms Dictionary (Code Cell)

Code:

```
```python
symptoms_dict = {}
'''
```

Explanation: Another empty dict for symptoms (maps symptom names to numbers).

18. Condition Calculator (Code Cell)

Code:

```
```python
for index, symptom in enumerate(x):
 symptoms_dict[symptom] = index
def calc_condition(exp,days):
 sum=0
 for item in exp:
 sum=sum+severityDictionary[item]
 if((sum*days)/(len(exp)+1)>13):
 print("You should take the consultation from doctor. ")
 else:
 print("It might not be that bad but you should take precautions.")
```
```

Explanation:

- Fills `symptoms\_dict` with symptom names as keys and their column positions as values.
- `calc\_condition` calculates if symptoms are serious. Sums severity scores, multiplies by days, and if >13, suggests seeing a doctor. Otherwise, take precautions.

19. Load Metadata (Markdown)

Code/Content: `# Load Metadata`

Explanation: Heading for loading extra data.

20. Get Descriptions (Code Cell)

Code:

```
```python
```



```

def getDescription():
 global description_list
 with open('MasterData/symptom_Description.csv') as csv_file:
 csv_reader = csv.reader(csv_file, delimiter=',')
 line_count = 0
 for row in csv_reader:
 _description={row[0]:row[1]}
 description_list.update(_description)
'''

```

**Explanation:** Function to load disease descriptions from CSV into `description\_list`. Reads each row and adds disease name (row[0]) and description (row[1]).

### ### 21. Get Severity (Code Cell)

Code:

```

'''python
def getSeverityDict():
 global severityDictionary
 with open('MasterData/symptom_severity.csv') as csv_file:
 csv_reader = csv.reader(csv_file, delimiter=',')
 line_count = 0
 try:
 for row in csv_reader:
 _diction={row[0]:int(row[1])}
 severityDictionary.update(_diction)
 except:
 pass
'''

```

**Explanation:** Loads symptom severities. Turns the score (row[1]) into an integer. `try-except` skips bad rows.

### ### 22. Get Precautions (Code Cell)

Code:

```
```python
def getprecautionDict():
    global precautionDictionary
    with open('MasterData/symptom_precaution.csv') as csv_file:
        csv_reader = csv.reader(csv_file, delimiter=',')
        line_count = 0
        for row in csv_reader:
            _prec={row[0]:[row[1],row[2],row[3],row[4]]}
            precautionDictionary.update(_prec)
    ...
```

Explanation: Loads precautions. Each disease gets a list of 4 precautions.

23. Get User Info (Code Cell)

Code:

```
```python
def getInfo():
 print("-----HealthCare ChatBot-----")
 print("\nYour Name? \t\t\t",end="->")
 name=input("")
 print("Hello, ",name)
 ...
```

**Explanation: Greets the user and asks for their name. Simple input/output.**

### ### 24. Pattern Checker and Second Predictor (Code Cell)

Code:

```
```python
def check_pattern(dis_list,inp):
```

```

pred_list=[]
inp=inp.replace(' ','_')
patt = f"{inp}"
regexp = re.compile(patt)
pred_list=[item for item in dis_list if regexp.search(item)]
if(len(pred_list)>0):
    return 1,pred_list
else:
    return 0,[]
def sec_predict(symptoms_exp):
    df = pd.read_csv('Data/Training.csv')
    X = df.iloc[:, :-1]
    y = df['prognosis']
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=20)
    rf_clf = DecisionTreeClassifier()
    rf_clf.fit(X_train, y_train)
    symptoms_dict = {symptom: index for index, symptom in enumerate(X)}
    input_vector = np.zeros(len(symptoms_dict))
    for item in symptoms_exp:
        input_vector[[symptoms_dict[item]]] = 1
    return rf_clf.predict([input_vector])
'''

```

Explanation:

- **`check\_pattern`**: Searches for symptoms matching user input (using regex). Returns matches or nothing.
- **`sec\_predict`**: A second prediction function. Reloads data, trains another Decision Tree, creates a "vector" (array of 0s/1s for symptoms), and predicts based on confirmed symptoms. Used for double-checking.

25. Print Disease (Code Cell)

Code:

```

```python
def print_disease(node):
 node = node[0]
 val = node.nonzero()
 disease = le.inverse_transform(val[0])
 return list(map(lambda x:x.strip(),list(disease)))
```

```

Explanation: From the Decision Tree's output, finds the predicted disease. ``nonzero`` gets the position, ``inverse_transform`` turns number back to name, and strips spaces.

26. Main Chatbot Logic (Code Cell)

Code: (This is the big one—the entire chatbot interaction.)

Explanation: This is the heart of the chatbot. It uses the Decision Tree to ask questions interactively.

- Asks for a symptom, checks if it matches (with ``check_pattern``), lets you choose if multiple.
- Asks how many days.
- Traverses the Decision Tree (``recurse`` function): Based on your symptom, it follows branches, collects more symptoms, and predicts.
- Asks yes/no for additional symptoms from ``reduced_data``.
- Uses ``sec_predict`` for a second opinion.
- Calculates condition severity.
- Prints the disease, description, and precautions.
- (Text-to-speech is commented out.)

At the end, it calls the load functions, gets user info, and runs the chatbot.

Notes on chat_bot.py

The ``.py`` file is just the notebook code pasted together as a script. It runs the same way but without cells. To run it, use ``python chat_bot.py`` in the terminal.

How to Run

- For the notebook: Open in Jupyter/VS Code, run cells in order.
- For the script: Install dependencies (`pip install -r requirements.txt`), then run.
- It needs the data files in Data and MasterData.